

第9回 リスト構造

これまでに学んだことの応用

前回のおさらい

- 構造体の作り方・使い方
- typedef
- 構造体の配列
- 構造体とポインタ
- 構造体をメンバとする構造体

第8回 演習問題 1

「第8回 演習課題1」

検証済

穴埋め問題 難易度: 1 目標タイム: 10分 プログラミング基礎 / 型基礎

問題文

このプログラムは、与えられた二つの座標を左上と右下の頂点とする矩形（長方形）の面積を求めるものである。整数型でx座標とy座標を定義する構造体pointと、二つのpoint構造体メンバーtop_leftとbottom_rightを持つ構造体rectangleを適切に定義して、プログラムを完成させなさい。

入力される値

この問題では入力から情報は与えられません。システムの都合上 None が与えられていますが、受け取る必要はありません。無視してください。

期待される出力値

矩形の4頂点の座標と矩形の面積

```
1  #include <stdio.h>
2
3  struct point{
4      int x;//[記入行]
5      int y;//[記入行]
6  };
7
8  struct rectangle{
9      struct point top_left;//[記入行]
10     struct point bottom_right;//[記入行]
11 };
12
13 int main(){
```

第8回 演習問題2

「第8回 演習課題2」

検証済

穴埋め問題 難易度: 1 目標タイム: 10分 プログラミング基礎 / 型基礎

問題文

このプログラムでは、都道府県の名前、人口、面積が'stat_t'という構造体で管理される。main関数で入力されるいくつかの都道府県全体の人口と面積からその地域の人口密度を表示するプログラムを書きなさい。

入力される値

いくつかの県の県名、人口、面積が入力されます。県名に0が入力されたところで入力を終了とします。

```
16 scanf( "%s", buf );
17 while( buf[0] != '0' ){
18     strcpy(
19         data[num].name // 【記入行】
20         , buf );
21     scanf( "%d %lf",
22         &(amp;data[num].population)// 【記入行】
23         ,
24         &(amp;data[num].area)// 【記入行】
25         );
26     num++;
27     scanf( "%s", buf );
28 }
```

```
30 for( int i = 0; i < num; i++ ){
31     sum_popl +=
32     data[i].population// 【記入行】
33     ;
34     sum_area +=
35     data[i].area// 【記入行】
36     ;
37 }
38 printf( "%d県の人口密度: %f\n", num, sum_popl / sum_area );
```

第8回 演習問題3

「第8回 演習課題3」

検証済

コーディング問題 難易度: 2 目標タイム: 20分 プログラミング基礎 / 型基礎

問題文

小銭最少化問題。

常に財布の中の小銭の枚数を最少（これを最適とする）にしたい。支払額に応じて、どの硬貨を何枚使えばいいかを求めるプログラムを完成させなさい。ただし、問題文に書かれたソースコードの一部を利用し、関数opt_paymentの中身を記述することで完成させなさい。また、最初の所持金額や枚数はこのソースコードに予め書かれた状態とする。

入力される値

支払いの金額

期待される出力値

Price(値段)

支払い前の所持硬貨の各枚数

支払い硬貨の各枚数

最適支払い後の所持硬貨の各枚数

```
1  #include <stdio.h>
2
3  typedef struct { // 硬貨の枚数 配列先頭から1円玉, 5円玉, 10円玉, ..., 500円玉とする
4      int ini_num[6]; // 初期の所持コイン枚数
5      int amount;      // 初期の所持金額
6      int pay_num[6]; // 支払いコイン枚数
7      int chg_num[6]; // お釣りコイン枚数
8      int opt_num[6]; // 支払い後の最適化されたコイン枚数
9  } coins;
10
11 void opt_payment( int price, int *value, coins *transaction ); // 最適支払い枚数
12 void show_coins_number( int *coins_num, int *value ); // 所持硬貨の各枚数を表示
13
14 int main(){
15     int price; // 値段 (単位 円)
16     int value[6] = {1, 5, 10, 50, 100, 500}; // 硬貨の価値 (単位 円)
17     coins transaction = {{3, 1, 4, 0, 2, 1},
18                          748,
19                          {0, 0, 0, 0, 0, 0},
20                          {0, 0, 0, 0, 0, 0},
21                          {0, 0, 0, 0, 0, 0}};
```

第8回 演習

「第8回 演習課題3」 検証済

コーディング問題 難易度: 2 目標タイム: 20分 プログラミング

問題文

小銭最少化問題。

常に財布の中の小銭の枚数を最少（これを最適とする）硬貨を何枚使えばいいかを求めるプログラムを完成させたソースコードの一部を利用し、関数opt_paymentの。また、最初の所持金額や枚数はこのソースコード

入力される値

支払いの金額

期待される出力値

Price(値段)

支払い前の所持硬貨の各枚数

支払い硬貨の各枚数

最適支払い後の所持硬貨の各枚数

```
35 void show_coins_number( int *coins_num, int *value ){
36     for( int i = 0; i < 6; i++ ){
37         printf( "%3d円x%1d枚 ", value[i], coins_num[i] );
38     }
39     printf( "\n" );
40 }

41
42 void opt_payment( int price, int *value, coins *transaction ){
43     int zankin;
44     zankin = transaction->amount - price;
45     for( int i = 5; i >= 0; i-- ){
46         transaction->opt_num[i] = zankin / value[i]; 支払い後の最適枚数
47         if( transaction->ini_num[i] >= transaction->opt_num[i] ){
48             transaction->pay_num[i] = transaction->ini_num[i] - transaction->opt_num[i];
49             } 使う枚数
50         else{
51             transaction->chg_num[i] = transaction->opt_num[i] - transaction->ini_num[i];
52             } お釣りの枚数
53         zankin -= value[i] * transaction->opt_num[i];
54     }
55     return;
56 }
```

第9回 リスト構造

これまでに学んだことの応用

アルゴリズムとデータ構造で学んだやつ

ポインタと構造体を学んだので、プログラムの中にデータ構造を持ち込むことができるようになった

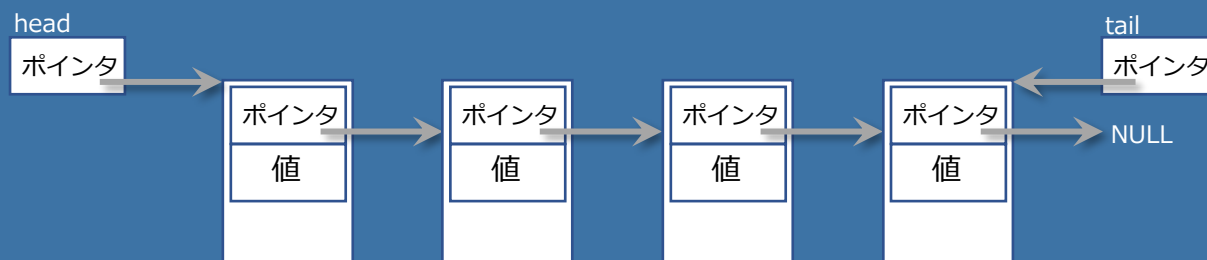
- 連結リスト、循環リスト・双方向リスト
- スタック、キュー
- 2分木、AVL木、B木

この中で今日はベースとなるリスト構造
(単方向リスト／双方向リスト)をやしましょう

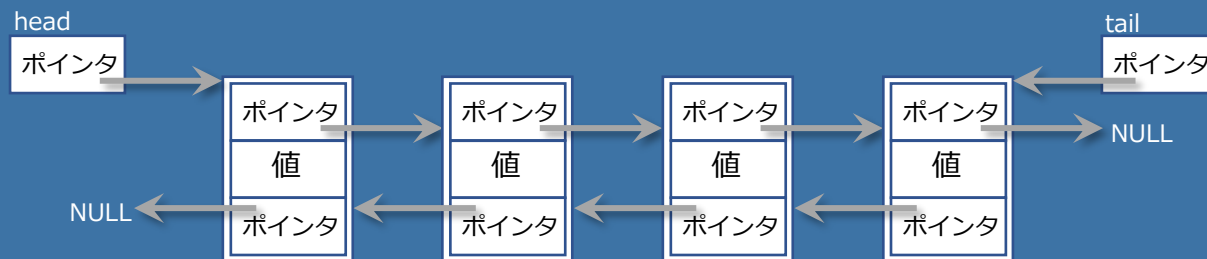
復習 連結リスト

1年生の時にやったのを覚えていると思うけど

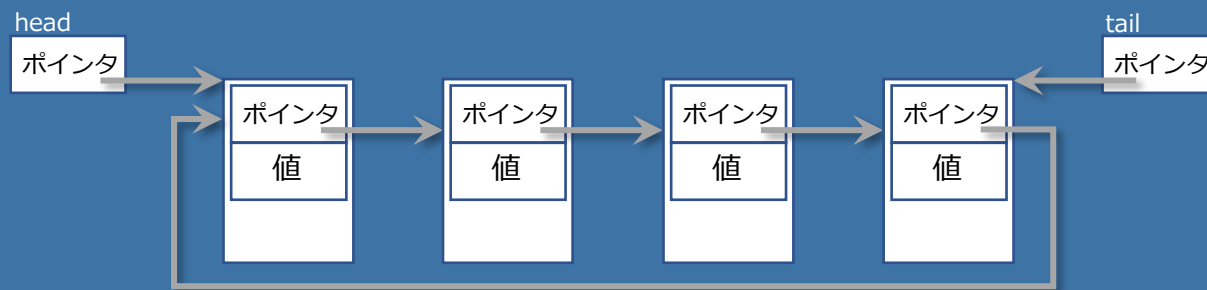
単方向リスト



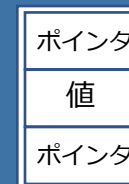
双方向リスト



環状リスト



ノード（構造体）



連結リストを実現するには

- ノードの挿入
 1. 新規メモリ確保
 2. ポインタ付け替え
- ノードの値の表示
- ノードの探索
- ノードの削除
 1. メモリ解放
 2. ポインタ付け替え

リスト構造用のノード構造体

ノードの構造体メンバは
そのノードに記憶する値と
隣接ノードへのポインタ（隣接ノードのアドレス）

```
typedef struct node {  
    int data;  
    struct node *next;  
    struct node *prev;  
} node;
```

ノード（構造体）



ノードの生成 (挿入)

main関数で

headとtailの準備

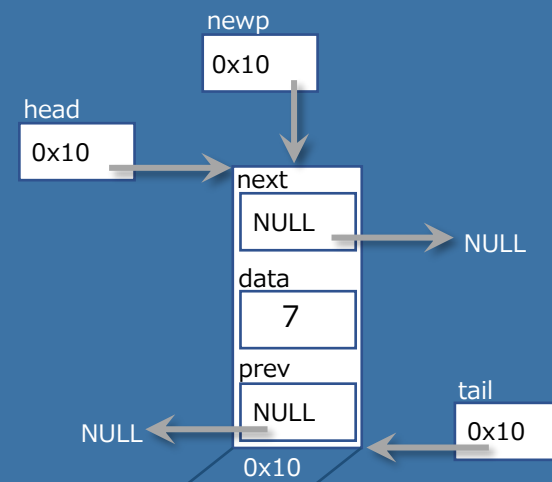
変数としてメイン関数に宣言

新ノードの挿入

メモリ確保

メンバを設定

隣接ノードのポインタ切替



```
int insert_node( int value, node **nextp, node **prevp ){
    node *newp;
    if( (newp = (struct node *)malloc( sizeof( struct node ) )) == NULL )
        return 0;    // メモリ確保失敗
    newp->data = value;
    newp->next = *nextp;
    newp->prev = *prevp;
    *prevp = newp;
    *nextp = newp;
    return 1;
}
```

```
int main( void ){
    node *head = NULL, *tail = NULL;
    if( insert_node( 7, &head, &tail ) == 0 ){
        return 0;
    }
}
```

malloc関数

動的なメモリ確保
ヒープ領域のメモリ割り当て

使い方

```
p = (int *)malloc( sizeof( int ) );
```

sizeof関数と型名を使って確保したいメモリサイズを確保

ポインタ変数pに確保メモリの先頭アドレスを代入（戻り値をその型のポインタにキャストして代入）

絶対に確保できるとは限らないので確保できなかったときは安全にプログラムや関数を終了するような書き方をするのが一般的

使い終わったら、
メモリを解放

```
free( p );
```

メモリ領域	記憶されるもの	特徴
プログラムメモリ	関数、定数、文字リテラル	書き込めない
スタックメモリ	ローカル変数	領域サイズ小さい 関数呼び出し時に作られ関数が終わるとき破棄
静的メモリ	大域変数、静的変数	変数の有効な範囲や寿命に注意
動的メモリ	メモリ確保関数で	欲しいときに欲しい分だけ 領域サイズ特大

```
if( (p = (int *)malloc( sizeof( int ) )) == NULL )  
    return 0;
```

ノードの生成 (挿入)

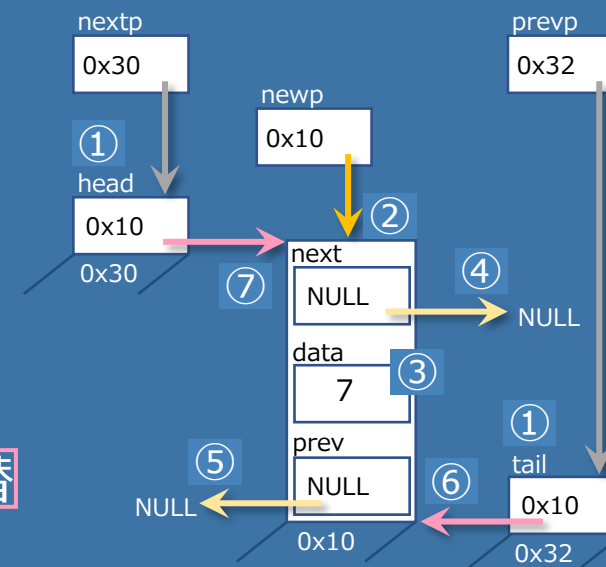
main関数で
headとtailの準備
変数としてメイン関数に宣言

新ノードの挿入

メモリ確保

メンバを設定

隣接ノードのポインタ切替



```
int insert_node( int value, node **nextp, node **prevp ){
    node *newp;
    ② if( (newp = (struct node *)malloc( sizeof( struct node ) )) == NULL )
        return 0; // 割り当て失敗
    ③ newp->data = value;
    ④ newp->next = *nextp;
    ⑤ newp->prev = *prevp;
    ⑥ *prevp = newp;
    ⑦ *nextp = newp;
    return 1;
}
```

```
int main( void ){
    ① node *head = NULL, *tail = NULL;
    if( insert_node( 7, &head, &tail ) == 0 ){
        return 0;
    }
}
```

切り替えたいところ

ノードの追加 (先頭にどんどん追加。スタックみたいに)

作業用ポインタの準備

新ノードの挿入

メモリ確保

メンバを設定

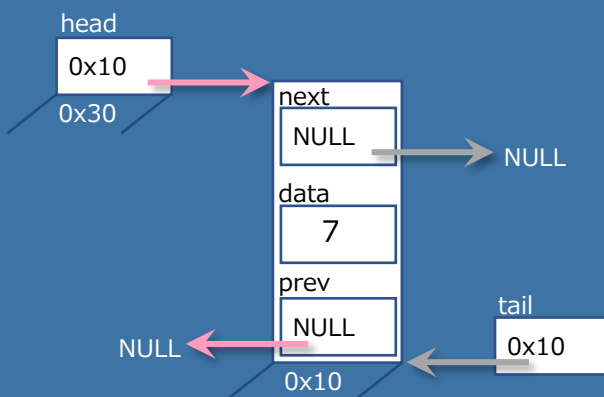
隣接ノードのポインタ切替

main関数内での呼び出し

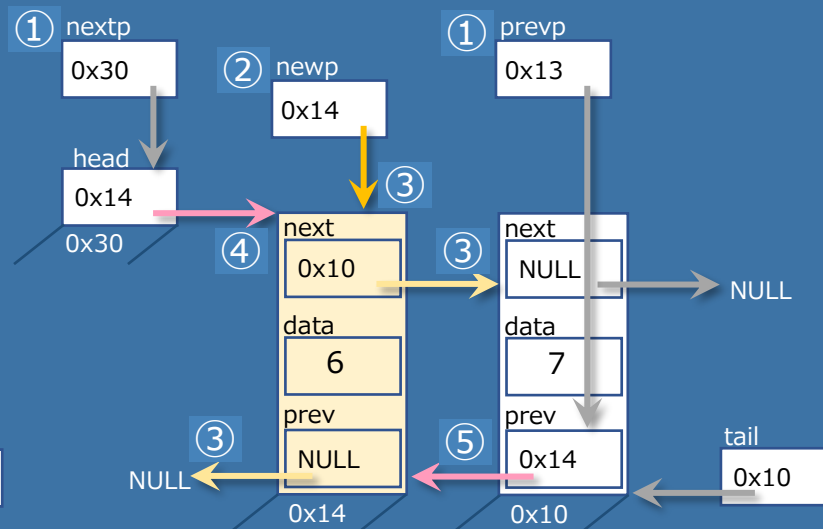
```
insert_node( 6, &head, &(head->prev) );  
insert_node( 3, &head, &(head->prev) );  
insert_node( 2, &head, &(head->prev) );
```

↑ ↑
切り替えたいところ

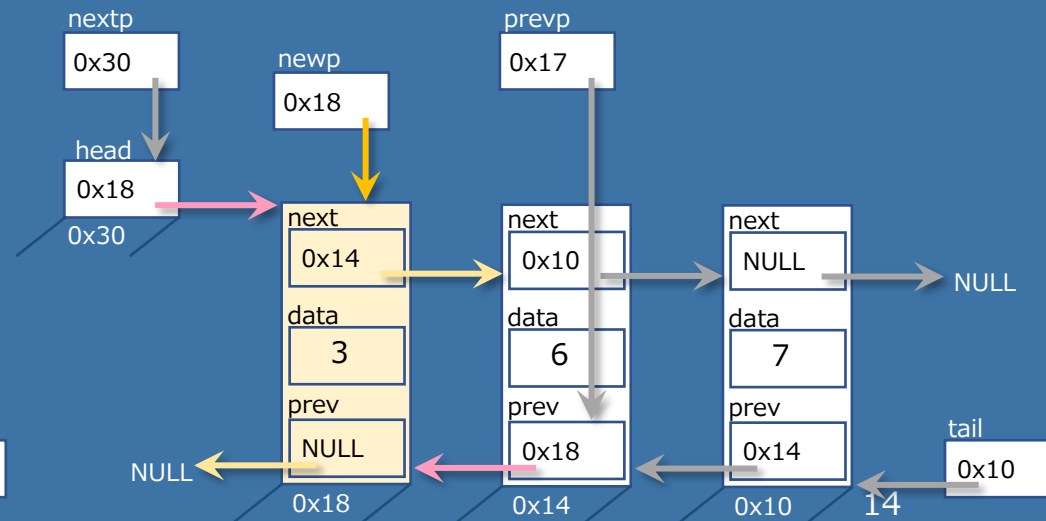
追加前



追加後



さらに追加後



ノードの追加 (最後尾に追加)

作業用ポインタの準備

新ノードの挿入

メモリ確保

メンバを設定

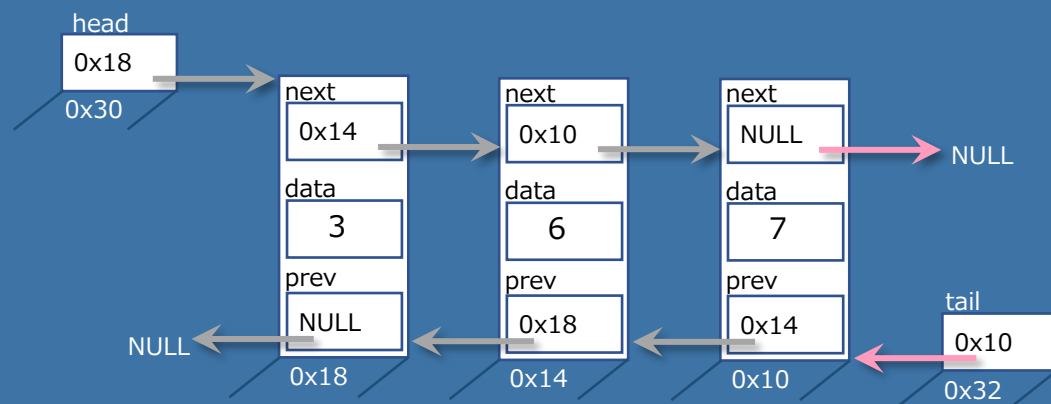
隣接ノードのポインタ切替

main関数内での呼び出し

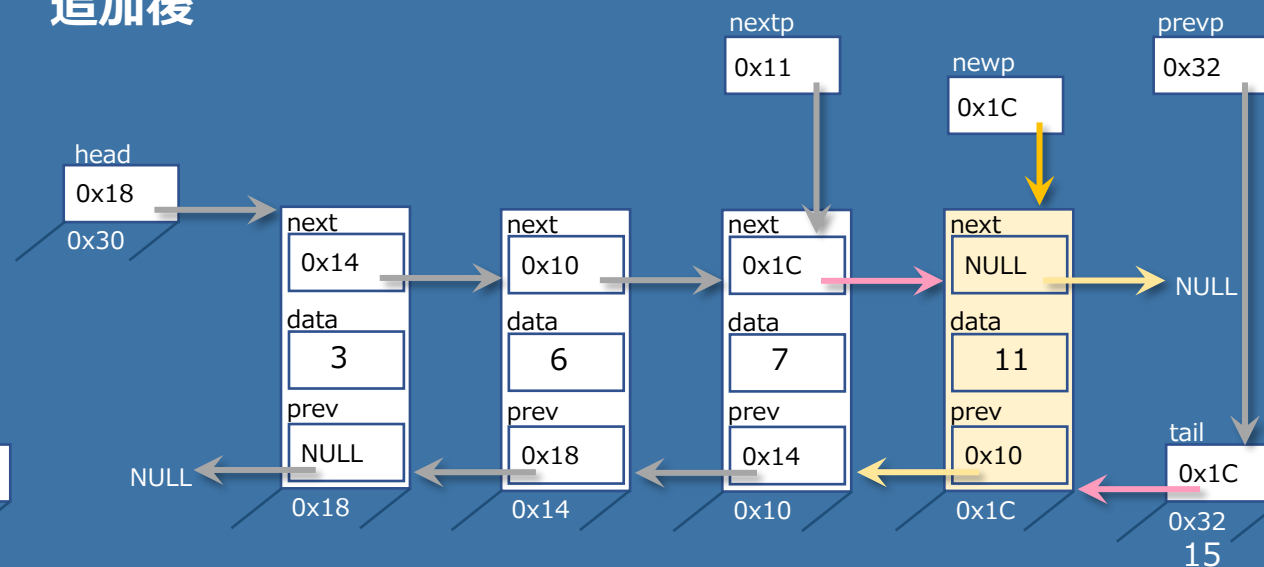
```
insert_node( 11, &(tail->next), &tail );
```

切り替えたいところ

追加前



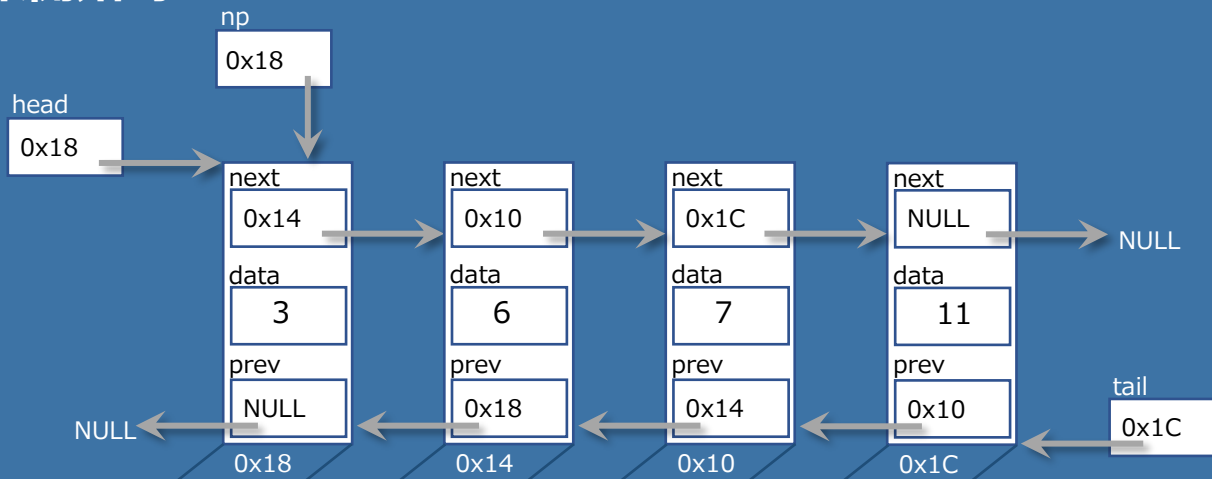
追加後



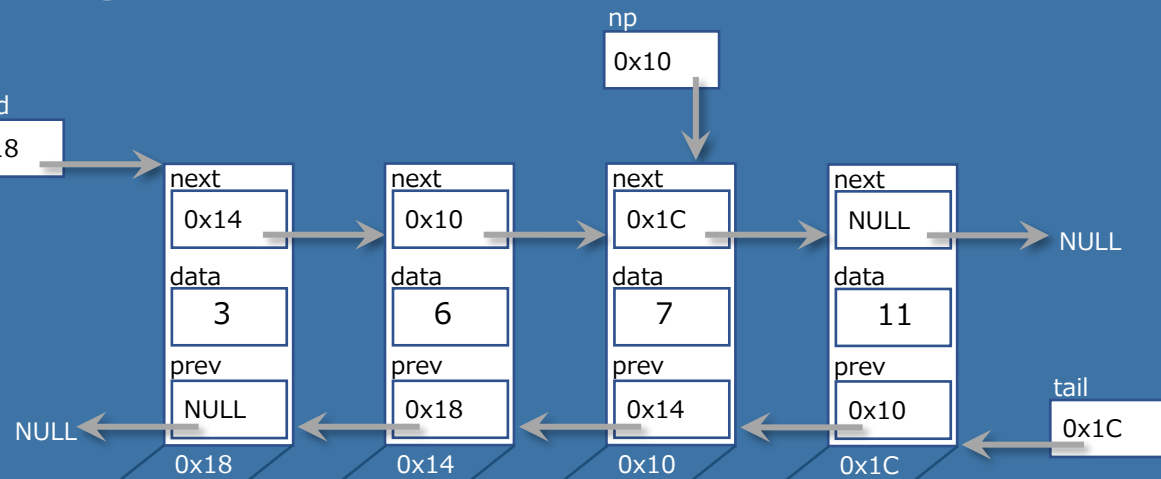
ノードの探索

headから順にたどって
変数valueの値と等しくな
ければポインタを先へ進める

探索開始時



探索終了時



```
node *search_node( int value, node *np ){  
    while( (np != NULL) && (value != np->data) )  
        np = np->next;  
    return np;  
}
```

main関数内での呼び出し

```
node *np;  
np = search_node( 6, head );
```

↑ npがNULLなら探索失敗
(その値はリストに存在しない)

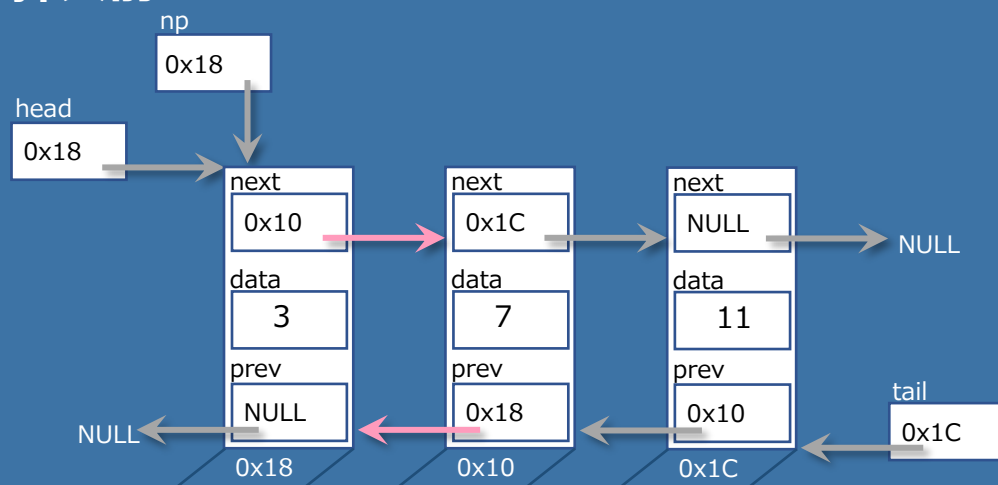
途中へのノードの挿入

挿入したい場所の前のノードにポインタを進めてinsert_node()

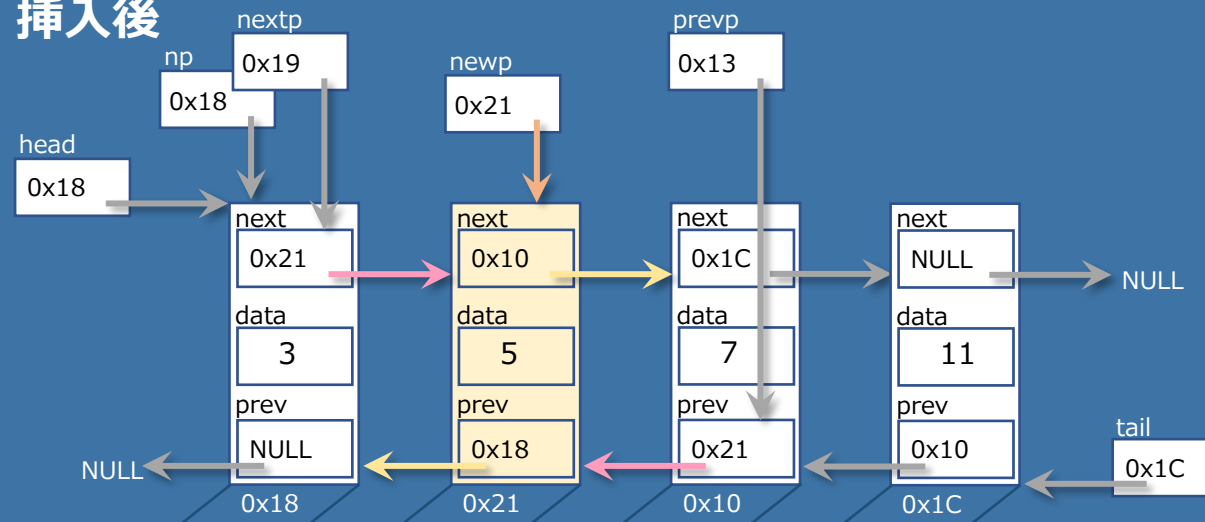
main関数内での呼び出し

```
np = search_node( 3, head ); // 挿入する直前のノードまでポインタを進めて  
insert_node( 5, &(np->next), &(np->next->prev) );
```

挿入前



挿入後



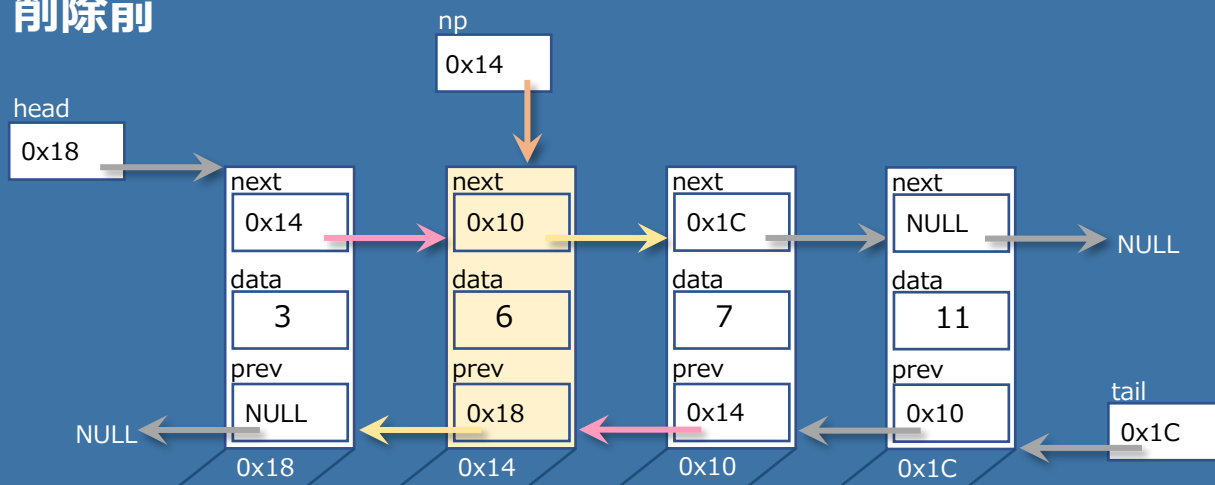
ノードの削除

隣接ノードのポインタ切替
メモリ解法

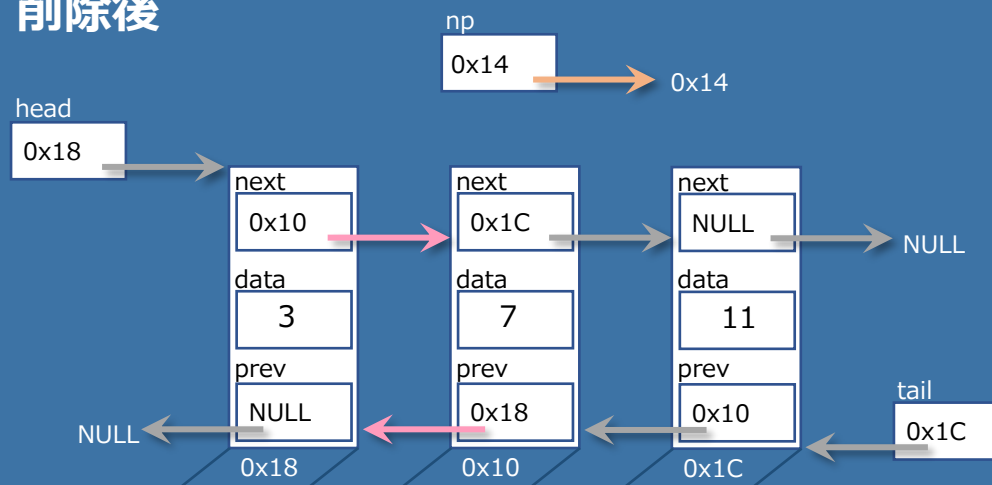
```
void delete_node( node *np, node **head, node **tail ){  
    np->prev->next = np->next;  
    np->next->prev = np->prev;  
    free( np );  
}
```

main関数内での呼び出し
`delete_node( np, &head, &tail );`

削除前



削除後



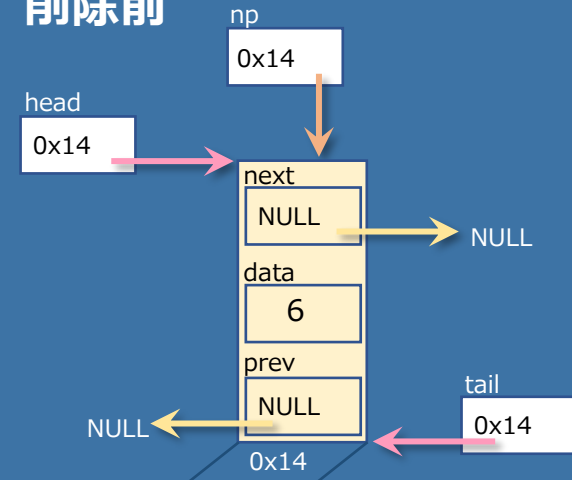
ノードの削除 (端のノードの考慮)

headやtailのポインタ切替が必要

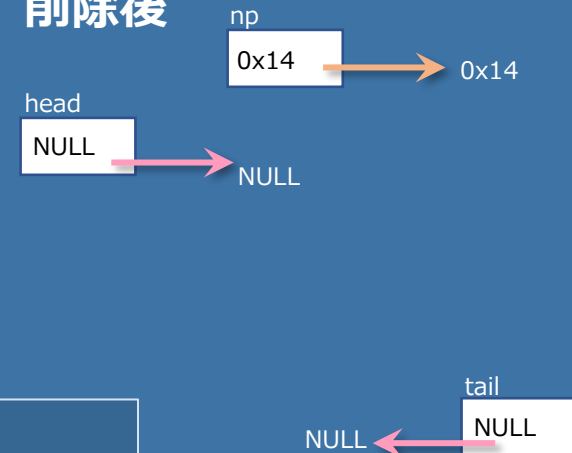
```
void delete_node( node *np, node **head, node **tail ){
    if( np == *head ){
        // 最左端ノードの削除の場合
        *head = np->next;
    }
    else{
        np->prev->next = np->next;
    }
    if( np == *tail ){
        // 最右端ノードの削除の場合
        *tail = np->prev;
    }
    else{
        np->next->prev = np->prev;
    }
    free( np );
}
```

main関数内での呼び出し
delete_node(np, &head, &tail);

削除前



削除後



今日おさえておくべき事柄

- リスト構造の作り方
- 探索・挿入・削除・値の表示
- ポインタと構造体を使えば色々なデータ構造をプログラムで扱えるようになる

演習課題

セッション	パス
第9回 演習課題 1 コード iSOPVmqw	765478
練習問題B コード mY7D2Tgl	619075

宿題

次回の予習

教科書 pp.132~139